COMPLETION

`.Translate(ctx,text,prompt)` Argument-Contract Audit — COMPLETION (§11.4.118)

Revision: 1 **Last modified:** 2026-06-16T15:43:07Z **Mode:** READ-ONLY, NON-COMMITTING. Sole committer is a separate agent; this agent staged/committed nothing (§11.4.84). Two temporary `_test.go` wire probes written under `pkg/coordination/` + `pkg/verification/`, run, then DELETED — `git status` confirmed clean of probe residue. **Method:** superpowers:systematic-debugging + §11.4.6 (FACT-only) + §11.4.118 + §11.4.69. **Predecessor:** `docs/qa/translate_arg_audit_20260616-153353/AUDIT.md` (the 5 confirmed-broken + 4 NEEDS-RUNTIME-CONFIRM baseline).

---

1. The 4 NEEDS-RUNTIME-CONFIRM sites — resolved verdicts (FACT)

#8 `pkg/coordination/multi_llm.go:445` — BROKEN (wrong-content / data-loss) in the ensemble/factory path

Evidence (static, FACT):

- `instance.Translator.Translate(ctx, text, contextHint)` (line 445). `instance.Translator` is `translator.Translator` built EITHER by `llm.NewLLMTranslator` (line 210, the per-provider discovery path → `*LLMTranslator`, **SAFE**, composes) OR by `c.translatorFactory(ctx)` (line 257, `initializeFromFactory`, the injected ensemble).
- The factory in production is `bridge.EnsembleFactory` → `ProviderDiverseTranslators` (`pkg/bridge/bridge.go:527/534`) which wraps each verified client in `&clientTranslator{client: c}`, and the only client `realClientBuild` builds is `llm.NewOpenAIClient` (`bridge.go:313-314`). `clientTranslator.Translate` delegates **VERBATIM** (`bridge.go:554-555`).
- `contextHint` originates from the ebook pipeline: `UniversalTranslator.translateChapter/ translateSection/ translateMetadata` call `TranslateWithProgress(ctx, <REAL CONTENT>, <LABEL>, …)` where the LABEL is a non-empty constant: `"Book title"`, `"Book description"`,

`"Chapter title"`, `"Section title"`, **`"Section content"`** (`pkg/translator/universal.go:158/173/199/231/246`). The `MultiLLMTranslatorWrapper` forwards that as `TranslateWithRetry(ctx, text, contextHint)` (`translator_wrapper.go:81`).
- Production wiring: `cmd/cli/main.go:361-364` builds `NewMultiLLMTranslatorWrapperWithFactory(..., b.EnsembleFactory(task))` and feeds it to `UniversalTranslator` → the factory ensemble path is LIVE for `cmd/cli` multi-llm.

Evidence (wire probe, FACT — captured this session): a probe drove the REAL `MultiLLMCoordinator` (`NewMultiLLMCoordinatorWithFactory`) with a faithful verbatim-delegating `clientTranslator`-equivalent over a real `OpenAIClient` pointed at an httptest server, `text="REAL_SECTION_CONTENT_THAT_MUST_BE_TRANSLATED"`, `contextHint="Section content"`:

```
#8 TranslateWithRetry: out="DUMMY_TRANSLATION"
server-side user message captured = "Section content"   -> WRONG-CONTENT (real text DROPPED)
```

The real section content NEVER reaches the LLM; only the label `"Section content"` is sent → the model "translates" the literal words "Section content" → catastrophic silent data loss for every chapter/section/title in the `cmd/cli` multi-llm ensemble path.

Correction to the predecessor audit: the baseline marked #8 "BROKEN iff ensemble AND `contextHint` empty." FACT: `contextHint` is NEVER empty from the ebook pipeline — it is always a non-empty label — so the failure is the **wrong-content** class (label sent, content dropped), not empty-payload, and it fires on EVERY ebook translation in the ensemble path. More severe than the baseline implied. The non-factory discovery path (`*LLMTranslator`, line 210) remains **SAFE** (composes).

#9 `pkg/coordination/multi_llm.go:529` — BROKEN (wrong-content / data-loss) in the ensemble/factory path

Same receiver (`inst.Translator`), same `contextHint`, same factory wiring as #8; the consensus fan-out variant (`TranslateWithConsensus`). **Wire probe (FACT):**

```
#9 TranslateWithConsensus: out="DUMMY_TRANSLATION"
server-side user message captured = "Section content"   -> WRONG-CONTENT (real text DROPPED)
```

Identical verdict and severity to #8.

#11 `pkg/api/server.go:196` — DEAD-CODE (no production wiring); SAFE-by-deadness

Evidence (FACT, §11.4.124 investigate-before-claiming):

- `s.translator` is populated ONLY by `Server.SetTranslator` (`server.go:119-120`).

- `grep -rn 'SetTranslator(' cmd/ pkg/ internal/` (excluding the method def): the ONLY callers are `pkg/api/server_test.go` (7 sites, every one passing `mockTranslator`). **Zero production callers.**
- `pkg/api.Server` is built only by `NewServer` (`server.go:51`); `grep` for `api.NewServer` / `api.Server{` across `cmd/ pkg/ internal/` (excluding tests): **empty — no production constructor.**
- The shipping servers use DIFFERENT types: `cmd/api-server/main.go` builds its own struct around `api.NewVerifierHandler` + a gin router (lines 212/219); `cmd/server/ main.go` uses `api.NewHandler` (line 136). Neither constructs `pkg/api.Server` nor calls `SetTranslator`.

Therefore `server.go:196` `s.translator.Translate(ctx, req.Text, "ru->sr")` is reachable ONLY from in-package tests, where `s.translator` is always a mock. It never reaches a real OpenAI/`clientTranslator` in any shipping path. **Verdict: DEAD-CODE** (latent wrong-content shape — non-empty `"src->tgt"` 2nd arg — but unreachable in production). Per §11.4.124 this is investigate-before-remove: do NOT delete on sight; if retained, it should be wired correctly OR removed in a separate, git-history-citing commit. `UNCONFIRMED:` nothing — the deadness is proven by exhaustive caller/constructor grep.

#22 `pkg/verification/notes.go:166` — SAFE (composes; clientTranslator type-unreachable)

Evidence (static, FACT):

- The `NoteTaker.translator` field is the **concrete** type `*llm.LLMTranslator` (`notes.go:140`), NOT the `translator.Translator` interface. `NewNoteTaker` takes `*llm.LLMTranslator` (`notes.go:145`); its only production callers (`pkg/verification/multipass.go:502/544`) pass a `*llm.LLMTranslator`.
- A `clientTranslator` (interface value) is **not assignable** to a `*llm.LLMTranslator` field — the verbatim-delegate path is structurally unreachable at #22. `*LLMTranslator.Translate` COMPOSES (`llm.go:364` `createTranslationPrompt(text, contextStr)` → `llm.go:388` `client.Translate(ctx, text, prompt)` sends the composed prompt as the 2nd arg).

Evidence (wire probe, FACT — captured this session): drove the REAL `NewNoteTaker` → `GenerateNotes(... location="Chapter 3", originalText, translatedText ...)` through a real `*llm.LLMTranslator` over httptest:

```
#22 server-side user message captured = a 2353-char COMPOSED
prompt
    (contains the literary note-taking instruction AND the
"Chapter 3" context),
    != "Chapter 3" -> the note instruction REACHES the LLM -> SAFE
```

Correction to the predecessor audit: the baseline marked #22 "BROKEN (wrong-content) in ensemble path — same as polisher, injector untraced." FACT: it is **SAFE** — notes.go has NO interface-typed ensemble field (unlike polisher's `ensemble map[string]translator. Translator`), so the verbatim clientTranslator is type-incompatible and never reached.

Honest §11.4.6 note (NOT the arg-contract bug class): the composed prompt embeds the note-taking instruction as the "Russian text:" to translate and appends "Serbian translation (Ekavica only):". The instruction is delivered (no data loss), but the note-taking use is structurally bolted onto the translation-prompt template — a SEMANTIC/quality oddity, separate from the data-loss arg-contract class this audit covers. Flagged for awareness, not counted as a wrong-content break.

---

2. FINAL COMPLETE confirmed-broken site list (FACT, wire-proven)

6 sites broken (was 5 in the baseline; #8+#9 promoted from NEEDS-CONFIRM to BROKEN; #22 demoted to SAFE; #11 resolved DEAD-CODE).

Empty-payload (empty user message) — 4 sites

#	site	wiring	proof
1	`cmd/markdown-translator/main.go:244`	I1 raw `BestClient`→OpenAIClient, 2nd arg `""`	baseline probe (A)
2	`pkg/preparation/coordinator.go:290`	clientTranslator ensemble path, 2nd arg `""`	baseline probe (A)
3	`pkg/preparation/coordinator.go:361`	same as #2	baseline probe (A)
4	`pkg/preparation/coordinator.go:424`	same as #2	baseline probe (A)

Wrong-content (1st-arg real content dropped, non-empty wrong label sent) — 3 sites

#	site	wiring	proof
5	`pkg/verification/polisher.go:563`	`ensemble` interface map → clientTranslator; 2nd arg `location`="Chapter N"	this session: userMsg=="Chapter 3", verification prompt dropped
8	`pkg/coordination/multi_llm.go:445`	factory ensemble → clientTranslator; 2nd arg `contextHint`="Section content"	this session: userMsg=="Section content", real content dropped
9	`pkg/coordination/multi_llm.go:529`	same as #8 (consensus variant)	this session: userMsg=="Section content", real content dropped

Resolved NON-broken

- **#11 `pkg/api/server.go:196` — DEAD-CODE** (no production constructor/ caller; mock-only in tests).
- **#22 `pkg/verification/notes.go:166` — SAFE** (concrete `*llm.LLMTranslator` field, composes).

Defense-in-depth `OpenAIClient.Translate` fix coverage (unchanged from baseline + this session): the prompt==""→text fallback + empty-message refusal covers the 4 EMPTY-PAYLOAD sites (#1-#4). It does **NOT** cover the 3 WRONG-CONTENT sites (#5,#8,#9): their 2nd arg is NON-EMPTY ("Chapter N" / "Section content"), so the `prompt==""` fallback never fires and the empty-guard never triggers — the raw client still sends the label and drops the real content. The wrong-content class needs a separate fix.

---

3. Wrong-content fix design (#5 polisher, #8/#9 multi_llm) + RED-test shape

Root cause (FACT)

The three wrong-content sites all call a **verbatim-delegating** `clientTranslator` (`bridge.go:554`) via the `translator.Translator` interface, passing real content in arg-1 and a label in arg-2. `clientTranslator.Translate` forwards arg-2 to `OpenAIClient.Translate` which sends ONLY arg-2 as the user message (`openai.go:155 Content: prompt`). The label becomes the user message; the real content is dropped.

The canonical correct convention already exists in the same package: `bridge.go:369 realInvokeDispatch` does `client.Translate(ctx, "", full)` — content in arg-2, arg-1 empty. The `clientTranslator` violates this by passing the caller's (text, contextStr) straight through without composing.

Recommended fix — make `clientTranslator` compose (single choke-point, fixes all 3)

`clientTranslator` is the SINGLE verbatim delegate that ALL three wrong-content sites reach (polisher ensemble, multi_llm factory retry, multi_llm factory consensus). Fixing it there fixes #5/#8/#9 at once and any future ensemble caller, without touching three call sites.

Change `pkg/bridge/bridge.go` `clientTranslator.Translate` (and its `TranslateWithProgress`) to COMPOSE a single prompt embedding BOTH the content (arg-1 `text`) and the context/label (arg-2 `contextStr`) into the 2nd arg per the bridge.go:369 convention, e.g.:

```
func (c *clientTranslator) Translate(ctx context.Context, text,
        contextStr string) (string, error) {
    // Compose: real content (text) MUST reach the model;
        contextStr is a label/hint,
```

```
    // never the payload. Mirrors
        *LLMTranslator.createTranslationPrompt + bridge.go:369.
    full := composeTranslationPrompt(text, contextStr) // content-
        bearing, contextStr as Context:
    out, err := c.client.Translate(ctx, "", full)
    ...
}
```

where `composeTranslationPrompt` builds a real translation/verification instruction that INCLUDES `text` as the body and `contextStr` as a labelled hint (NOT as the body). This makes `clientTranslator` behave like `*LLMTranslator` (the already-SAFE path), so the ensemble path stops dropping content.

Caveat / alternative (honest §11.4.6): #5's caller (polisher `verifyWithLLM`) passes a *verification* prompt as arg-1 (`createVerificationPrompt`), not raw text — so a generic "translate this" composition is semantically wrong for polisher. The robust fix is therefore the **caller-side** convention fix: have the polisher (and any verification caller that already holds a complete instruction) pass the full instruction in arg-2 and `""` in arg-1 (the bridge.go:369 convention), while #8/#9 (which pass raw ebook content) get the compose-in-clientTranslator fix. Concretely:

- **#8/#9 (raw content + label):** compose in `clientTranslator` (content into the prompt body, label as Context:) — OR route the multi_llm ensemble through a `*LLMTranslator`-style composing wrapper instead of the verbatim `clientTranslator`.
- **#5 (already-composed verification prompt + location):** change polisher.go:563 to `translator.Translate(ctx, "", prompt+"\n\nLocation: "+location)` (content in arg-2), matching bridge.go:369 — the location becomes a labelled hint inside the single user message, and the verification instruction is no longer dropped.

Both directions converge on the invariant: **the content-bearing string MUST land in the arg the raw client actually sends (arg-2); a label/location/hint must be embedded, never substituted for the content.**

RED-test shape (§11.4.115 reproduce-on-broken-artifact + polarity switch)

One test source per fixed seam, `RED_MODE=1` reproduces the defect on the CURRENT (pre-fix) code, `RED_MODE=0` becomes the standing GREEN regression guard (§11.4.135). Use the httptest-capture pattern proven this session (no API key, deterministic):

```
Setup: httptest server records the user message; build the REAL
production wiring
       (NewMultiLLMCoordinatorWithFactory with a clientTranslator-
equivalent over an
       OpenAIClient at the test server, for #8/#9;
NewBookPolisherWithFactory for #5).

Drive: #8  coord.TranslateWithRetry(ctx, REAL_CONTENT, "Section
content")
       #9  coord.TranslateWithConsensus(ctx, REAL_CONTENT,
"Section content", 1)
```

```
       #5  bp.verifyWithLLM(ctx, "openai", ORIGINAL, TRANSLATED,
"Chapter 3")

Assert (RED_MODE=1, pre-fix -> MUST FAIL the GREEN assertion =
reproduce the defect):
   capturedUserMsg == "Section content" / "Chapter 3"   (the
label/location)
   AND capturedUserMsg does NOT contain REAL_CONTENT / the
verification instruction.

Assert (RED_MODE=0, post-fix -> GREEN guard):
   capturedUserMsg CONTAINS REAL_CONTENT (for #8/#9) or the
verification instruction
   (for #5)   - i.e. the content-bearing string actually reached
the model;
   AND for the empty-content negative: an all-whitespace text
yields an explicit error
   (composes the §11.4.69 empty-message refusal), never a silent
label-only send.
```

The assertion the RED test must make — "the polish/notes/translation request actually sends the verification/translation instruction AND the real content, not just the label" — is exactly what the two probes captured this session, so the RED reproduction is already demonstrated end-to-end (userMsg=="Chapter 3" / "Section content" on the pre-fix tree). Register each as a permanent guard (§11.4.135) in the same commit as the fix.

---

4. §11.4.6 honesty boundary

- **PROVEN FACT (wire repro this session):** #8 + #9 send the label "Section content" and drop the real ebook content in the factory ensemble path (`cmd/cli` multi-llm); #5 sends "Chapter 3" and drops the verification prompt; #22 sends the COMPOSED 2353-char note prompt (SAFE); #11 has zero production constructor/caller (DEAD-CODE). Probes built the REAL coordinator/polisher/note-taker over httptest — not stubs of the units under test.
- **Static FACT:** receiver-type analysis (#22 concrete `*llm.LLMTranslator` vs #5 interface `ensemble`); factory→clientTranslator→OpenAIClient-only chain; `contextHint` label origin from `universal.go`.
- **No UNCONFIRMED items remain** — all 4 NEEDS-RUNTIME-CONFIRM sites are settled with FACT. (The #22 semantic-oddity note is flagged as a separate quality concern, explicitly NOT the arg-contract data-loss bug class.)
- **Non-committing:** two probe `_test.go` files written, run, DELETED; `git status` verified clean of probe residue; packages rebuild green after removal. Nothing staged or committed by this agent.